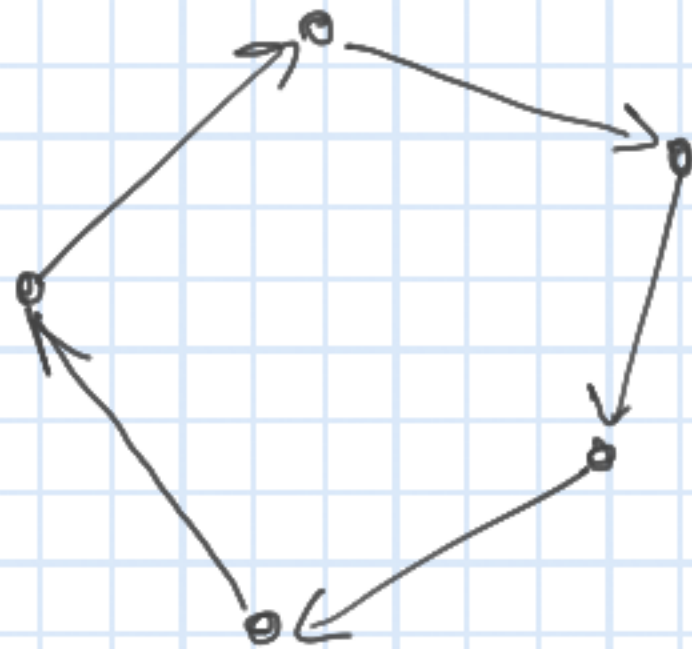


November 12, 2021

Cycle detection

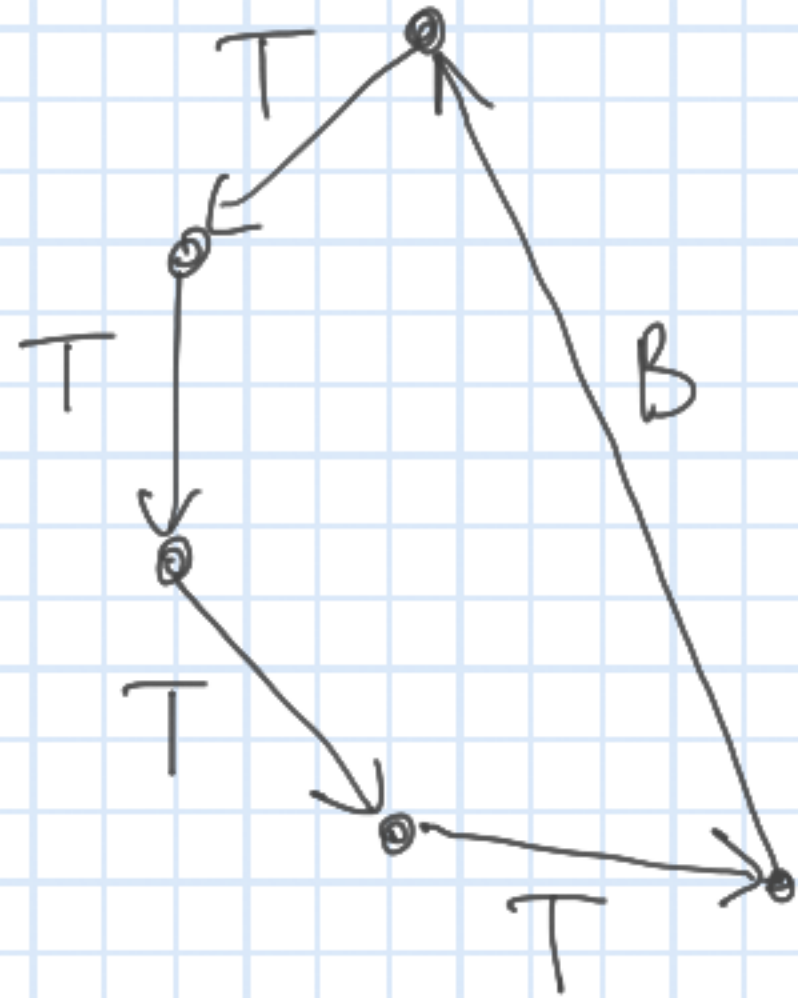
We are given a directed graph $G = (V, E)$.

Decide whether G contains a directed cycle



Observation

Run DFS. If you find a back edge, then it will form a cycle with the tree edges



Algorithm

Run DFS

→ if you find a back edge, then the graph contains a cycle (clear)

→ if you cannot find a back edge, then the graph is "cycle-free"
OR acyclic (less clear)

Notion

DAG - Directed Acyclic Graphs

DAGs form a special but very important subfamily of directed graphs

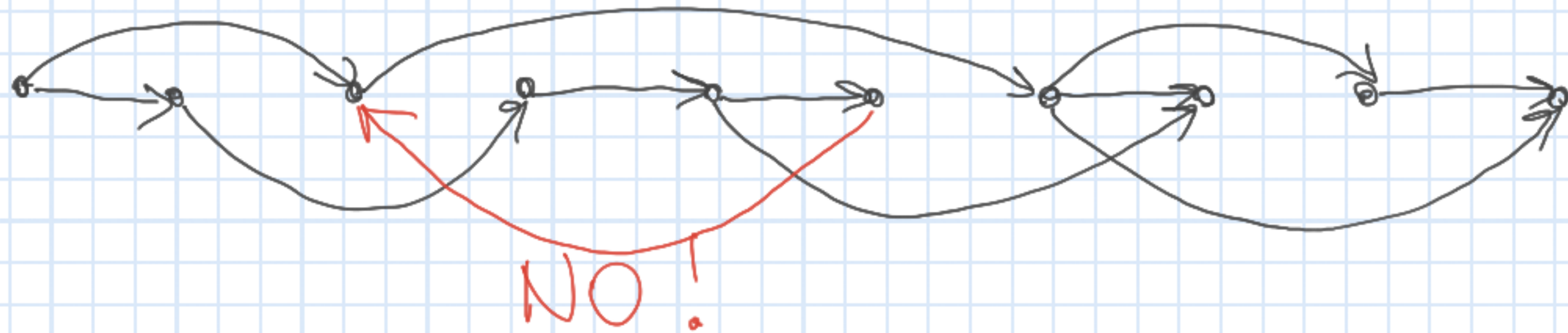
Many "hard problems" can be solved efficiently in this subfamily

Topological sorting (ordering)

A topological ordering of a directed graph $G = (V, E)$ is a permutation $(v_1, v_2, \dots, v_n)$ of V such that if $(v_i, v_j) \in E$, then $i < j$.

Visually

We can draw the graph in such a way that the vertices are on a straight line and each edge is "from left to right"



Theorem

A graph has a topological ordering $\Leftrightarrow$ it is a DAG

Algorithm that finds a topological ordering
(if exists)

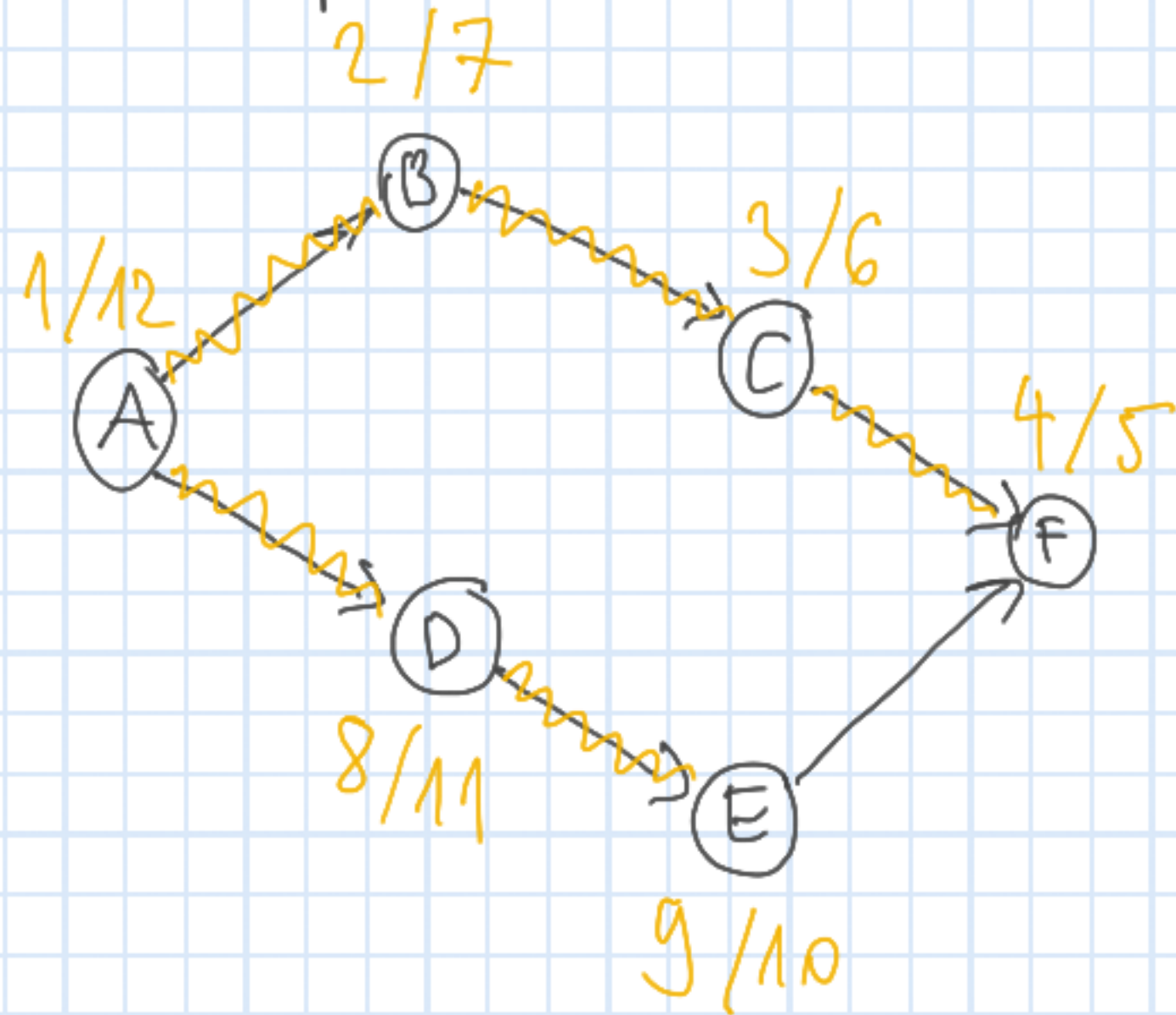
Run DFS

→ if you find a back edge, then no
topological ordering exists

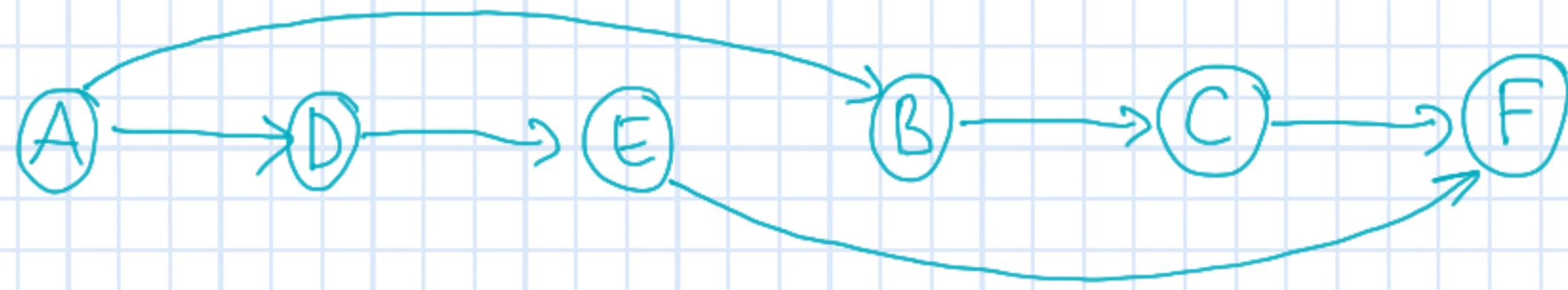
→ if not, list the vertices decreasingly
according to their finishing time,
the result is a topological ordering

If G is given by its adjacency lists, then this algorithm is linear ($O(|V| + |E|)$ time)

Example

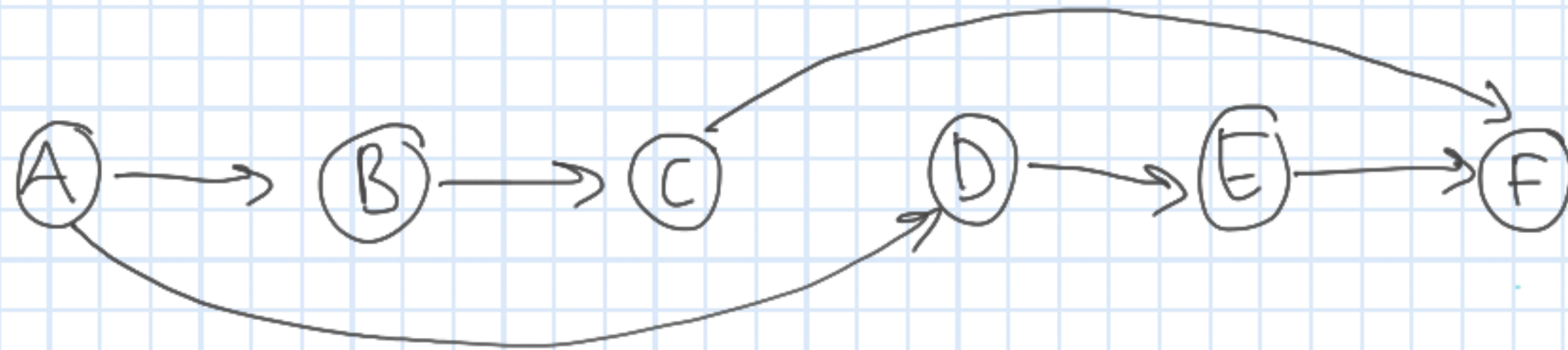


- ① Run DFS and find a topological ordering
- ② Find all topological orderings



topological
ordering

Another one :



Hw. : Find all !

Provide a brief explanation that
no more exists

Minimum cost spanning trees

Let $G = (V, E)$ be an undirected, connected graph.

A connected, cycle free subgraph $G' = (V, E')$ of G is called a spanning tree of G .

(A spanning tree contains all vertices of G)

We can define the cost of a spanning tree as the sum of the weights of its edges.

(This is not the only possibility)

Problem

Determine a minimum cost spanning tree of G .

Open problem

Can we find an $O(|V| + |E|)$ time deterministic algorithm to determine a MST (minimum cost spanning tree) of a graph $G = (V, E)$ given by its adjacency lists.

NOBODY KNOWS !!!

How many spanning trees can have a connected graph?

Cayley theorem

The maximum number of spanning trees of a connected graph with n vertices is n^{n-2}

(complete graph with different edge weights)

This implies that the naïve (brute force) method is inefficient

Nevertheless, there are efficient algorithms to determine an MST

→ Prim algorithm

→ Kruskal algorithm

Time complexity: $O(|E| \log |E|)$ (if the graph is given by its adjacency lists)

Prim algorithm

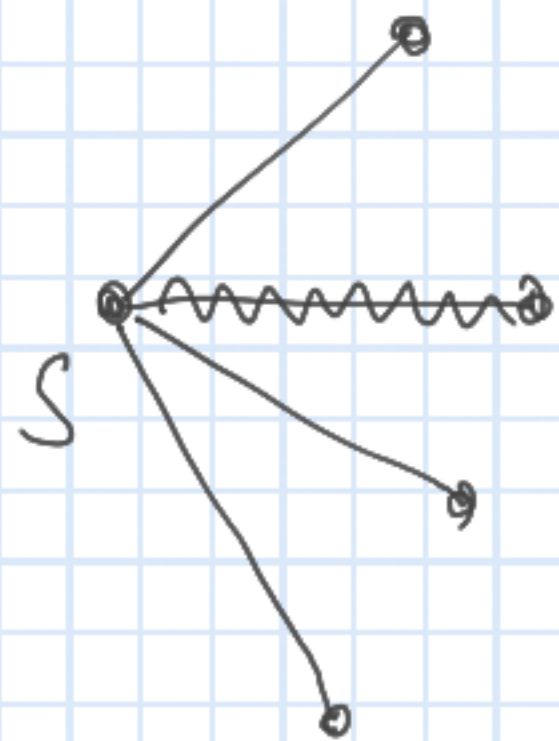
$G = (V, E)$ connected graph, given by
adjacency lists

$$W : E \rightarrow \mathbb{R}$$

$s \in V$ (starting vertex)

We grow a spanning tree from s in the
following way:

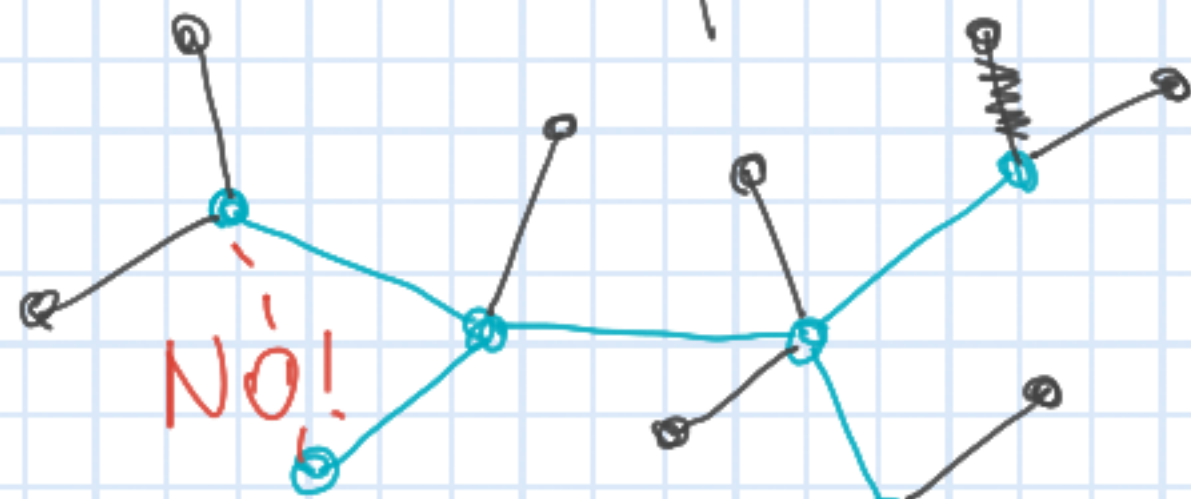
First phase :



select a min weight edge
adjacent to s

→ first edge of MST

General phase :



select a min weight edge
adjacent to one (but not two)
vertex of this

We are here in constructing in MST

→ next edge of MST

Pseudocode

We will store the edges of the MST in F ;

initially $F = \emptyset$

We will store the vertices of the MST in X ;

initially $X = \{s\}$

Prim (G)

$F := \emptyset$, $X := \{s\}$

while $X \neq V$ do

 let (u, v) be a min weight edge s.t. $u \in X, v \in V \setminus X$

$F := F \cup \{(u, v)\}$, $X := X \cup \{v\}$

Implementation issue

How can we select efficiently a min weight edge between X and $V \setminus X$

Idea :

Maintain a priority queue (heap) containing the edges between X and $V \setminus X$ (keys $\sim$ weights)

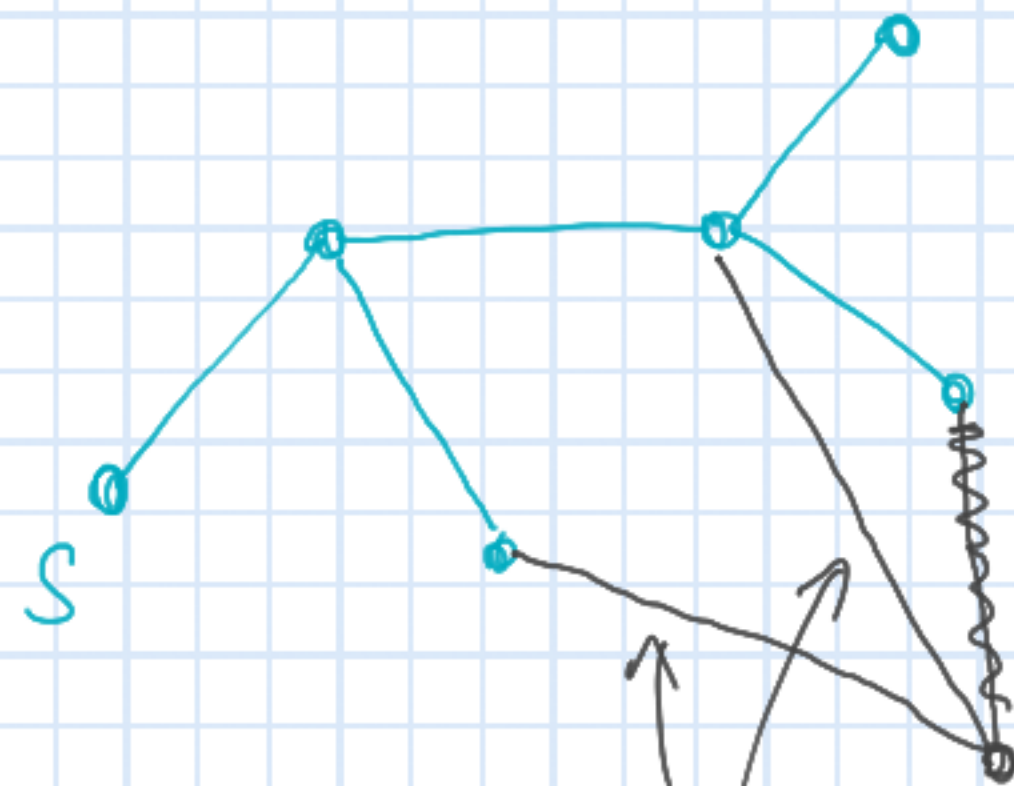
Initially this heap contains the edges adjacent to s

Selecting a min weight edge between X and $V \setminus X$

can be done by one or more MINDELETE,
and after selecting (u, v) we INSERT into
the heap the edges adjacent to v

WARNING!

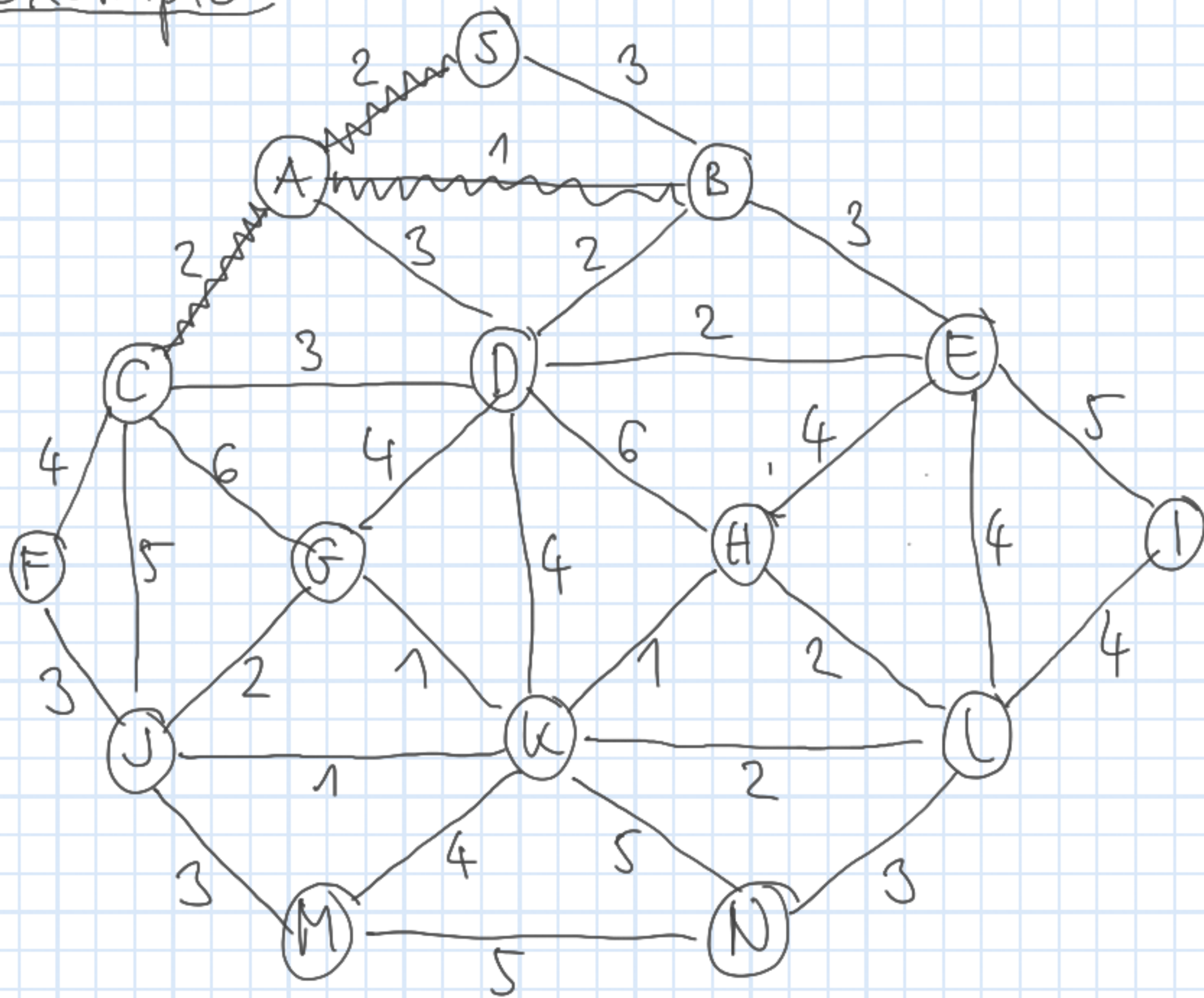
Sometimes one MINDELETE is not enough
because we cannot delete edges from the
heap whose both endpoints belong to X
after adding a new vertex to X :



new edge of the MST

they remain in the heap after
selecting this

Example



Heap

~~SA~~, SB

SB, ~~AB~~, AC, AD

SB, ~~AC~~, AD, BD, BE

SB, AD, BD, BE, CD,

CF, CJ, CG

⋮